ODME USER MANUAL

Operational Design Domain Modeling Environment

A detailed guide to the current ODME desktop application, including project setup, domain and scenario workflows, behavior tooling, ODD generation, sample generation, execution support, and packaged app usage.

Quick reading strategy: if you are new to ODME, start with **Main Window Tour**, then read **Domain Modelling**, and only after that move on to **Scenario**, **Behavior**, and **ODD** sections.

Product: Operational Design Domain Modeling Environment (ODME)

Manual revision: April 9, 2026

Main focus: how to use the shipped desktop application and the workflows available in its menus

Where to open this manual in the app: Help → User Manual

Contents

1. Purpose and scope
2. What ODME stores and generates
3. Launching the application
4. Main window tour
5. Recommended end-to-end workflow
6. File menu and project lifecycle
7. Domain Modelling workflows
8. Scenario Modelling workflows
9. Behavior-related workflows
10. ODD generation and ODD Manager
11. Scenario Manager, execution, and plugins
12. Artifacts on disk
13. Troubleshooting and practical advice

1. Purpose and Scope

ODME is a desktop modeling environment for building and maintaining **Operational Design Domain (ODD)** models with the **System Entity Structure (SES)** formalism, a formal tree to describe all possible variants of your system. In one application you can create a structural domain mode, add node metadata such as variables, constraints, distributions, and behaviors, derive scenario-specific versions, and generate supporting outputs for downstream analysis and automation

This manual focuses on the current ODME application as shipped in the repository. The screenshots and descriptions match the current UI where feasible, but exact spacing or panel sizing may vary slightly by operating system or display scaling.

2. What ODME Stores and Generates

Artifact	What it represents	Why it matters
Project XML and graph XML	The main model structure and graph layout.	These are the core files the editors use to reconstruct the model.
`.ssdvar`, `.ssdcon`, `.ssdbeh`, `.ssdflag`	Serialized metadata for variables, constraints, behaviors, and editor state.	Without them, the tree or graph may load but side metadata may be missing.
Schema and XML outputs	Generated XML and schema material derived from the model.	Used by ODD generation, exports, and some downstream workflows.
ODD snapshots	Saved ODD manager datasets under the project `odd/` directory.	Lets you preserve and export generated ODD views.

Artifact	What it represents	Why it matters
CSV sample files	Generated sample sets from ODD ranges or YAML-based workflows.	Useful for testing, batch scenario creation, and automation.
Scenario XML files	Concrete scenario outputs, often generated from CSV rows.	These are convenient handoff artifacts for later simulation or processing.

3. Launching the Application

Packaged desktop application

1. Open the ODME GitHub-repository https://github.com/aeronautical-informatics/ODME/tree/main.
2. Click “Actions” on the menu bar and select “ODME CI”.
3. Select the latest successful push and scroll down to “Artifacts”.
4. Download
   - “ODME-linux-app-image” for Linux or
   - “ODME-macos-app-image” for macOS or
   - “ODME-windows-app-image” for Windows

   and launch the executable file.
   Keep the whole application folder together; do not copy only the executable.

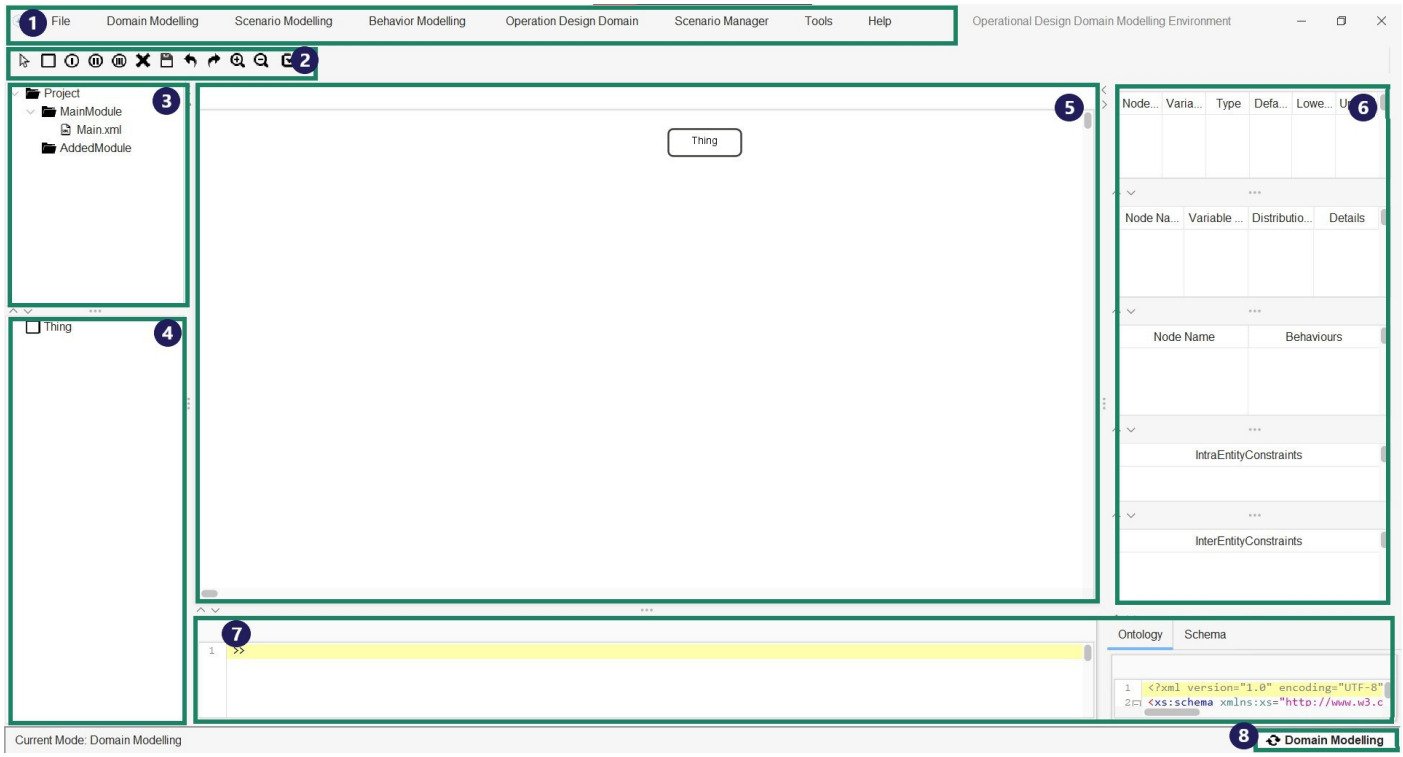
Run from source

1. Build the repository with Maven.
2. Run the shaded JAR or launch `odme.odmeeditor.Main` from your IDE.

> If you change code or documentation and the packaged app still looks old, rebuild the packaged app image. Recompiling Java classes alone does not update the packaged desktop folder automatically.

4. Main Window Tour

ODME is organized around a synchronized tree-and-graph workspace. Most work begins in the center graph canvas, but the left and right panels matter just as much because they control structure browsing and metadata editing.



Annotated ODME main window. The numbered callouts correspond to the legend below.

Callout	Area	What you use it for
1	Menu bar	Global actions such as project operations, ODD tools, scenario tools, and help.
2	Toolbar	Fast graph editing actions such as add node modes, save, undo, redo, and zoom.
3	Project Explorer	Shows project files and modules so you can verify the currently loaded project layout.
4	Model Structure tree	Hierarchical structure view of the SES model; right-click here for tree-based actions.
5	Graph canvas	The main visual modeling workspace for entities, aspects, specializations, and pruning work.
6	Node metadata panels	Shows variables, behaviors, constraints, and related data for the currently selected node.
7	Bottom viewers and console	Schema, ontology, XML, and console output used during validation, generation, and debugging.
8	Mode switch/status area	Shows the active mode and lets you switch between Domain Modelling and Scenario Modelling.

5. Recommended End-to-End Workflow

Step 1: create or open a project in **Domain Modelling**.

Step 2: build the structural SES tree on the graph canvas.

Step 3: add variables, constraints, distributions, and behaviors to the relevant entities.

Step 4: save the project and inspect generated XML or schema views. The corresponding files can all be in the application folder under your current projects folder.

Step 5: use ODD generation, ODD Manager, sample generation, execution, or plugins once the model is ready.

Step 6: switch to **Scenario Modelling** only after the base model is stable.

Step 7: create scenarios manually or optionally using Automatic Scenario Creation.

Step 8: structure and specify the characteristics of behaviors for your scenarios.

> The biggest source of confusion in ODME is starting downstream tooling too early. If something feels inconsistent, go back to the base model and verify the project structure before editing scenarios or generating ODD content.

6. File Menu and Project Lifecycle

Save

Saves the current project state and related generated outputs. Use this frequently when structural changes or metadata changes have been made.

Save As

Creates a renamed version of the project. ODME updates the internal project references, generates the required files in the new project location, and saves the tree and graph.

Save as PNG

Exports the current graph canvas as an image. This is useful for reports or quick reviews.

Exit

Closes the application.

Practical project lifecycle

- Use **Domain Modelling** → **New Project** for clean starts.

- Use **Domain Modelling** → **Open** to load an existing project folder.
- Use **Save** before switching modes or opening ODD tooling.
- Use **Save As** before major experimental changes if you want a checkpoint.

7. Domain Modelling Workflows

Domain Modelling is the primary authoring mode. It is where you create the SES structure that everything else depends on.

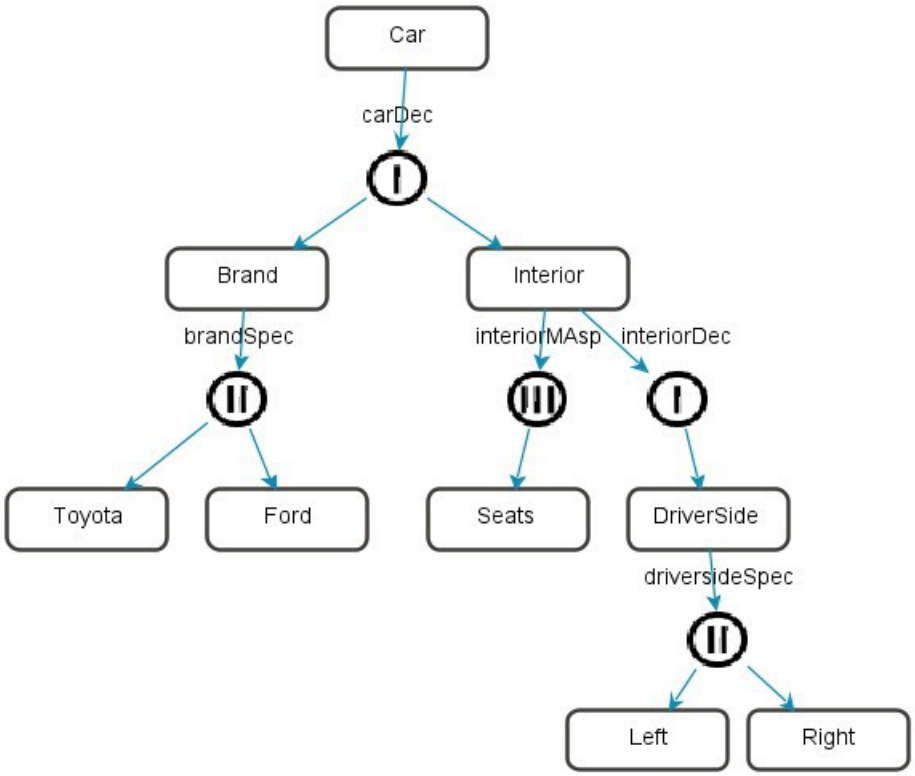
7.1 Start a project

1. Choose **Domain Modelling** → **New Project** to create a fresh model, or **Open** to load one.
2. Confirm that the project name and structure appear in the left Project Explorer.
3. Confirm that the model tree and graph canvas both show the same base root.

7.2 Add structure on the graph canvas

You can create nodes either from the toolbar or from graph context menus.

- **Blank canvas right-click**: add structural SES nodes such as entity, specialization, aspect, or multi-aspect.
  - **Entity**: A real or artificial system component which is an object of interest. (e.g. “Car”)
  - **Specialization**: A specific variation a thing can take (e.g. “Toyota” / “Ford” as the brand)
  - **Aspect**: The different parts of an entity. (e.g. “Brand”, “Interior”, etc.)
  - **Multi-Aspect**: When multiple child nodes of the same type are required, use a Multi-Aspect node.
- **Toolbar add nodes**: place new nodes directly on the graph by choosing an add mode and then clicking.
- **Connect nodes**: connect child nodes to a parent so the tree and graph stay synchronized.
- **Rename**: double-click or use the node context menu rename action.
- **Delete branch**: remove the selected node and its subtree when needed.



A possible domain model for a car with a few general characteristics.

Node naming consistency matters. ODME relies on suffix conventions such as `Spec`, `Dec`, and `MAsp` for certain flows, exports, and pruning operations.

7.3 Use the tree view for structure edits

The left Model Structure tree mirrors the graph. Right-clicking in the tree is useful when you want to work hierarchically rather than spatially.

- **Add Node**: insert a child under the selected tree node.
- **Add Variable**: add a variable directly through the tree workflow.
- **Delete Node**: remove a node from the tree and the synchronized graph.

7.4 Add metadata to entities

Most metadata editing happens by selecting an entity node in the graph.

- **Add Variable**: right-click an SES graph node and choose **Add Variable**.

- **Add Behaviour**: right-click an SES graph node and choose **Add Behaviour**.
- **Add Constraint**: available on aspect-related nodes where constraint logic is valid.
- **Edit from right-side tables**: select the node to refresh the data panels, then double-click an entry to edit it.

7.5 Delete or update metadata

Metadata	How to add it	How to change it	How to delete it
Variable	Node context menu or tree popup	Double-click variable entry in the variable table	Node context menu: delete one variable or delete all variables
Behaviour	Node context menu: **Add Behaviour**	Double-click the behavior table entry	Node context menu: **Delete Behaviour** or **Delete All Behaviours**; leaving the edited text empty also deletes that behavior
Constraint	Constraint-related node actions	Constraint table edit flow	Delete through the node context menu or the constraint table workflow

7.6 Import and export support

- **Import Template**: load a reusable project template into ODME.
- **Save as Template**: export the current model as a reusable XML-oriented template.

Domain Modelling: Specify the fundamental structure and characteristics of your system.

8. ODD Generation and ODD Manager

The **Operation Design Domain** menu is used after the structural model and metadata are in good shape.

What is an OD?

An Operational Domain (OD) describes the real-world environmental conditions, independent of your subsystem's capabilities.

What is an ODD?

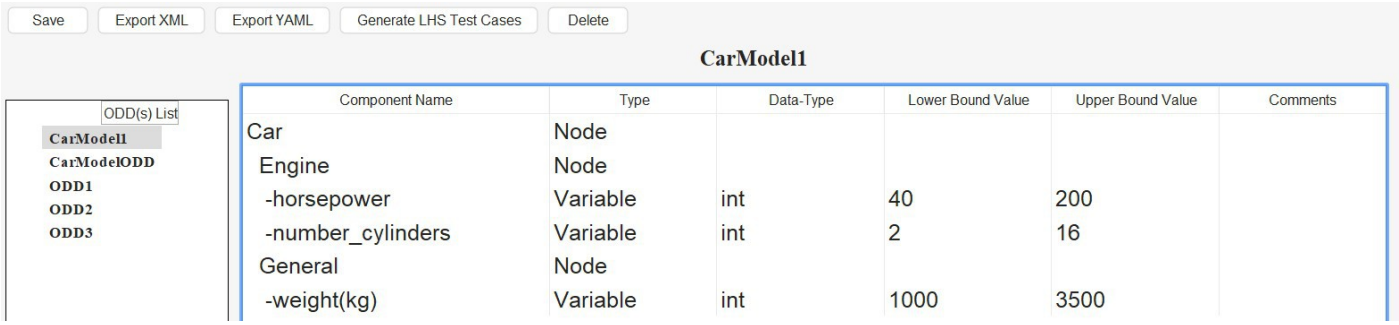
An Operational Design Domain (ODD) describes the specific conditions and boundaries under which your Subsystem is designed to operate.

Generate OD

Opens the current ODD representation derived from the project and prepares the data needed by the manager and export workflows.

ODD Manager

- Inspect saved ODD snapshots.
- Edit or review generated ODD content.
- Export ODD data to XML or YAML.
- Use ODD ranges as the basis for sample generation workflows.



Example of an ODD in the ODD Manager



```
▼<entity xmlns:vc="http://www.w3.org/2007/XMLSchema-versioning"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:noNamespaceSchemaLocation="ses.xsd" name="Car">
 ▼<aspect name="carDec">
  ▼<entity name="Engine">
     <var name="horsepower" type="int" default="none" lower="40" upper="200"> </var>
     <var name="number_cylinders" type="int" default="none" lower="2" upper="16">
     </var>
   </entity>
  ▼<entity name="General">
     <var name="weight(kg)" type="int" default="none" lower="1000" upper="3500">
     </var>
   </entity>
 </aspect>
</entity>
```

Corresponding XML file of the ODD shown above.

```
Car:
  Engine:
  - horsepower:
    type: int
    min: 40
    max: 200
  - number_cylinders:
    type: int
    min: 2
    max: 16
  General:
  - weight(kg):
    type: int
    min: 1000
    max: 3500
```

Corresponding YAML file of the ODD shown above.

OD / ODD: Keep track of your systems limitations and update them if needed to.

9. Scenario Modelling Workflows

Scenario Modelling is used after the base SES model is ready. In this mode the focus is no longer on designing the full domain tree, but on deriving and refining scenario-specific structures.

9.1 Switch into Scenario Modelling

1. Save the project first.
2. Use the mode switch in the bottom-right area to change the active mode.
3. Create or open a scenario using the scenario menu options.

9.2 Create or open a scenario

- **Scenario Modelling** → **Save Scenario**: create a named scenario and write it into the project scenario store.
- **Scenario Manager** → **Scenarios List**: open an existing scenario from the project.

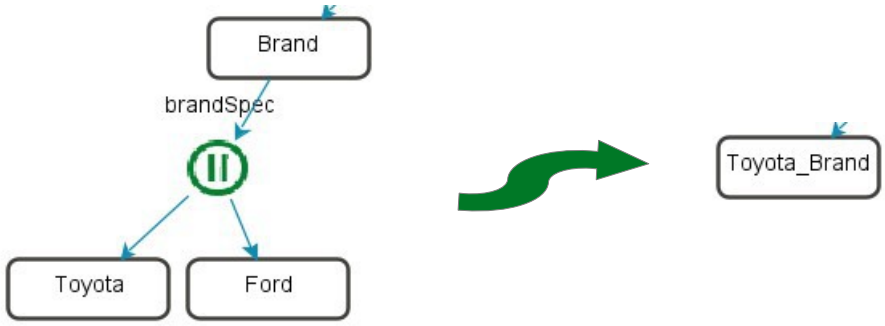
9.3 Prune the model

In scenario mode, ODME lets you prune branches from the structural model to create a scenario-specific version. This is how you move from a full SES description to a more concrete PES/scenario representation.

The nodes that are eligible to be pruned will be highlighted green and by right-clicking them, the bar “Prune it” will appear.

Depending on the selected node, three types of pruning are possible:

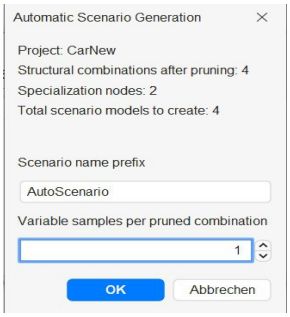
1. **Entity Variable Pruning:** Specific values can be assigned to entity variables.
2. **Specialization Node Pruning:** One specific of several possible child nodes can be selected.
3. **Multi-Aspect Node Pruning:** The number of instances of the child node can be defined.



Example of Specialization Node Pruning.

9.4 Automatic Scenario Creation

Use the Automatic Scenario Creation to efficiently create a set number of scenarios. Click “Automatic Scenario Creation in the menu bar of the Scenario List.



Automatic Scenario Creation tool

The number of Scenarios per automatically pruned combination depends on the number of variable Samples which can be selected with “Variable samples per pruned combination”.

> If a prune result looks wrong, return to Domain Modelling and fix the base structure first. Scenario Modelling works best when it is derived from a clean, stable project model.

Scenario Modelling: Set up specific scenarios that will be the starting point of your simulations.

10. Behavior-Related Workflows

ODME supports behavior data in two places: as behavior metadata attached to entities in the main domain model, and as a dedicated behavior modeling workflow that lets you build behavior trees for scenario-linked entities.

10.1 Entity behaviors in Domain Modelling

1. Open a project in Domain Modelling.
2. Right-click an entity node in the SES graph.
3. Choose **Add Behaviour** to attach a behavior name to that entity.
4. Select the entity again to refresh the right-side behavior table.
5. Double-click a behavior entry to update it.
6. To delete one behavior, either use **Delete Behaviour** from the entity context menu or clear the text in the edit dialog.
7. To delete all behaviors for an entity, use **Delete All Behaviours**.

10.2 Sync Behaviour from the menu

The **Behavior Modelling** → **Sync Behaviour** action opens the scenario list for available behavior work. This is the entry point to the separate behavior tree editor.

10.3 Behavior tree editor

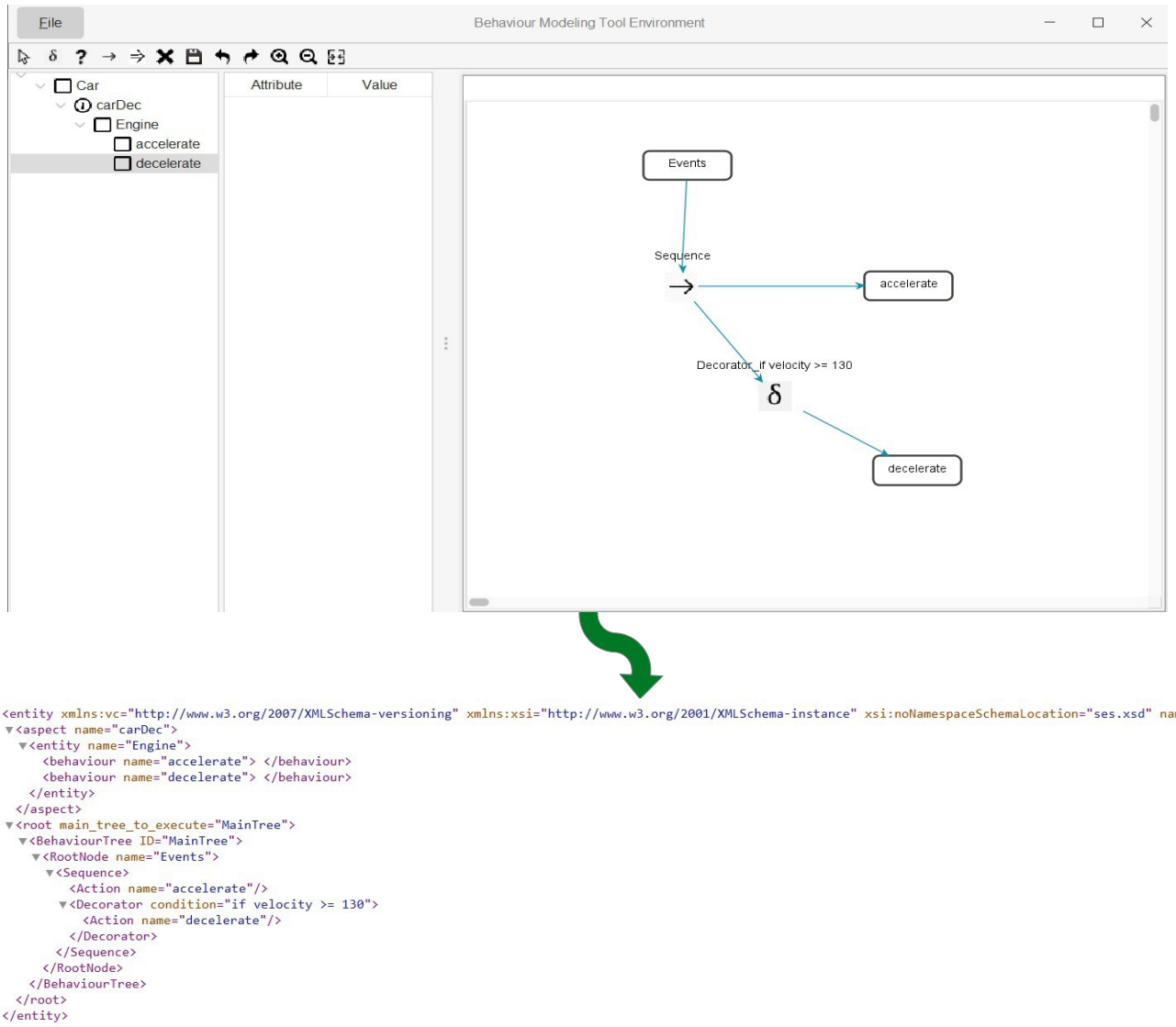


Annotated Behavior tree editor window. The numbered callouts correspond to the legend below.

Callout	Area	What you use it for
1	Toolbar	Fast graph editing actions such as undo, redo, merge, etc. or add decorators, selectors, sequences and parallel nodes.
2	Behavior tree	Hierarchical structure view of the SES model, including its behaviors.
3	Behavior attribute panel	Shows names and values of the attributes of the selected behavior.
4	Graph canvas	The main visual modeling workspace for defining the behavior's structure and characteristics

10.4 Behavior graph actions

- Select a leaf behavior in the left behavior tree, then place it into the graph.
- Add decorators, selectors, sequences, and parallel nodes from the toolbar.
  - **Decorator**: Specify a certain condition for a behavior to happen; double-click the decorator node to set or change its condition.
  - **Sequence**: Structure successive behaviors.
  - **Parallel**: Structure concurrent behaviors.
- Right-click an entity behavior node in the behavior graph to add behavior attributes.
- Select a behavior entity in the graph to populate the attribute table in the left panel.
- Use **Save Graph** to persist the behavior graph and generate behavior XML.
- Use **Merge** to merge behavior output with the XSD-related XML flow. The resulting XML-file will be called "MergedXMLforxsd.xml" and can be found under the current scenario's folder.




```
▼<entity xmlns:vc="http://www.w3.org/2007/XMLSchema-versioning" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="ses.xsd" name="Car">
  ▼<aspect name="carDec">
    ▼<entity name="Engine">
       <behaviour name="accelerate"> </behaviour>
       <behaviour name="decelerate"> </behaviour>
     </entity>
   </aspect>
  ▼<root main_tree_to_execute="MainTree">
    ▼<BehaviourTree ID="MainTree">
      ▼<RootNode name="Events">
        ▼<Sequence>
           <Action name="accelerate"/>
          ▼<Decorator condition="if velocity >= 130">
             <Action name="decelerate"/>
           </Decorator>
         </Sequence>
       </RootNode>
     </BehaviourTree>
   </root>
 </entity>
```

Behavior Tree and its corresponding XML file

Behavior modelling depends on having valid behavior data and scenario context already available. If the behavior editor opens but looks empty, check that the project and scenario contain the expected `.ssdbeh` data.

Behavior Modelling: Specify the structure and characteristics of behaviors that will take place while the system is running.

11. Scenario Manager, Execution, and Plugins

Scenario Manager menu

- **Scenarios List**: open and manage project scenarios.
- **Execution**: open the execution window for XML/YAML/Python-oriented workflows.
- **Feedback Loop**: reserved for scenario-related workflow extensions.
- **Generate Samples**: launch the constrained sample generation panel.

Generate Samples

This workflow supports YAML-based definitions, Latin Hypercube Sampling for continuous variables, constraint-aware acceptance logic, and CSV output suitable for downstream use.

Generate Scenario from CSV

Under **Save Scenario**, ODME includes **Generate Scenario** → **From CSV**. This converts CSV rows into individual scenario XML files.

Execution window

- Load XML or YAML content.
- Translate model outputs into Python-friendly data structures.
- Load or edit Python scripts.
- Run scripts and review console output.

Run Python Plugin

- Choose a Python script under **Tools** → **Run Python Plugin...**.
- ODME passes the current project context to the plugin runner.
- Output appears in the application console.

12. Artifacts on Disk

Location	Typical contents
Project root	Main project XML, graph XML, and serialized sidecar files such as `.ssdvar`, `.ssdcon`, `.ssdbeh`, and `.ssdflag`.
Scenario folders	Scenario-specific XML, behavior, and generated files.
`odd/` inside the project	Saved ODD manager outputs.
Generated CSV output	Sampling results used for scenario generation or external analysis.

> When copying a project between machines or ODME versions, copy the full project folder. Partial copies are the most common reason for models loading without their variables, behaviors, or constraints.

13. Troubleshooting and Practical Advice

The packaged application does not show the latest code or manual

- Rebuild the packaged app image, not only the Java classes.
- Open the freshly generated application folder, not an older copy elsewhere on disk.

Help → User Manual opens nothing

- Make sure the packaged app still contains the bundled `docs/manual.pdf` resource.
- If PDF opening fails, ODME falls back to the bundled HTML manual.
- Check that the desktop environment allows the app to open local files in the system viewer.

The model opens but variables, behaviors, or constraints are missing

- Verify that the expected `.ssd*` sidecar files are present next to the project data.
- Check that you opened the correct project root folder.
- If the issue appears in behavior mode, verify the scenario-specific `.ssdbeh` file path as well.

Behavior editor looks empty or incomplete

- Make sure the selected scenario actually has synchronized behavior data.
- Check that behaviors were attached to entities first from the domain model when required.
- Save the project before opening behavior workflows.

Advanced tooling feels harder than the base model editor

That is normal. ODD generation, sample generation, execution, and plugins make much more sense after the structural model and scenario workflows are already comfortable.

Final Advice

ODME works best when used in layers:

1. first the structural model
2. then node metadata
3. then ODD and downstream automation
4. then scenarios

If you keep that order, the menus, editors, and generated artifacts line up naturally. If anything becomes confusing, go back one layer and verify the foundation first.